ROTEIRO DE ESTUDOS – 2º BIMESTRE

Linguagens Formais e Compiladores

Organize a revisão pelos tópicos abaixo. A ênfase deve estar em compreender os conceitos, relacioná-los às fases de um compilador e aplicá-los em pequenos exemplos de reconhecimento, tokenização, análise sintática e construção de representações internas.

1. Hierarquia de Chomsky e gramáticas sensíveis ao contexto — Slide 8

- Revisar as classes Tipo 3, Tipo 2, Tipo 1 e Tipo 0 e a ideia de aumento do poder expressivo.
- Entender o papel das gramáticas sensíveis ao contexto (CSG) e a noção de regras dependentes do contexto.
- Produções não contrativas: em geral, o lado direito não deve ser menor que o lado esquerdo.
- Exemplo $a^n b^n c^n$: compreender por que a igualdade entre três blocos exige poder maior que o de uma GLC.
- Relacionar linguagens sensíveis ao contexto a autômatos linearmente limitados.
- Ênfase: classificar situações e justificar por que determinado modelo é ou não suficiente.

2. Linguagens recursivas, reconhecibilidade e decidibilidade — Slide 8

- Linguagem recursiva: existe algoritmo que sempre termina e responde sim ou não.
- Diferença entre linguagem recursiva e recursivamente enumerável.
- Conceito de problema decidível.
- Problema da parada como exemplo clássico de indecidibilidade.
- Exemplos decidíveis: pertencimento em linguagens regulares e em linguagens livres de contexto.
- Ênfase: distinguir 'decidir' de apenas 'reconhecer'.

3. Arquitetura e fases do compilador — Slide 8

- Front-end, middle-end e back-end.
- Análise léxica, análise sintática e análise semântica.
- Tabela de símbolos e sua relação com nomes, tipos e escopos.
- Representação intermediária (IR), otimizações e geração de código.
- Montador, ligador e carregador como etapas complementares após a geração de código.
- Ênfase: saber ordenar as fases e explicar a função principal de cada uma.

4. Análise léxica: token, lexema e padrão — Slide 9

• Token como categoria, lexema como texto concreto e padrão como regra que descreve lexemas válidos.

• Tokenização de linhas de código com identificadores, números, operadores, delimitadores e palavras reservadas.

• Diferença entre palavras reservadas e identificadores por consulta a uma tabela de palavras-chave.

• Espaços, tabulações, quebras de linha e comentários: normalmente não viram tokens do parser, mas precisam ser reconhecidos.

• Ênfase: tokenizar exemplos e explicar cada categoria encontrada.

5. Expressões regulares, AFN, AFD e regra do maior lexema — Slide 9

• Padrões léxicos definidos por expressões regulares.

• Relação entre expressões regulares e autômatos finitos.

• Conversão conceitual ER → AFN → AFD e por que o AFD é conveniente para scanners.

• Regra do maior lexema e prioridade entre regras, especialmente em operadores compostos como >=.

• Fluxo básico do scanner: ler, avançar estado, memorizar último estado final e emitir token ou erro.

• Ênfase: interpretar padrões e reconhecer situações de ambiguidade lexical.

6. Erros léxicos e recuperação — Slide 9

• Caracteres inesperados, strings não fechadas, números malformados e símbolos fora do alfabeto.

• Importância de registrar linha e coluna no token.

• Estratégias: token especial de erro, avanço de um caractere ou descarte até ponto de sincronização.

• Evitar mensagens em cascata e preservar a análise restante.

• Ênfase: formular diagnósticos claros com localização e causa provável.

7. Análise sintática descendente e LL(1) — Slide 10

• Análise sintática como verificação da estrutura da sequência de tokens segundo uma GLC.

• Descida recursiva: uma função por não terminal.

• Derivação mais à esquerda.

• Significado de LL(1): leitura da esquerda para a direita, derivação à esquerda e um token de lookahead.

• FIRST e FOLLOW em nível conceitual.

• Recursão à esquerda e necessidade de reescrita para analisadores descendentes.

• Ênfase: explicar como o parser escolhe produções e por que certas gramáticas precisam ser adaptadas.

8. Análise sintática ascendente — Slide 10

• Partir dos tokens e reduzi-los até o símbolo inicial.

• Deslocamento (shift) e redução (reduce).

• Pilha e reconhecimento de trechos correspondentes ao lado direito de produções.

• Analisadores LR e conflitos shift/reduce ou reduce/reduce.

• Ênfase: comparar o sentido da construção com a análise descendente.

9. Árvores sintáticas, precedência e ambiguidade — Slide 10

• Árvore sintática concreta: registra a aplicação das produções da gramática.

• Nós internos como não terminais e folhas como terminais.

• Precedência em expressões como 2 + 3 * 4.

• Ambiguidade sintática: mais de uma árvore possível para a mesma entrada.

• Ênfase: interpretar a estrutura da árvore e explicar como precedência altera o agrupamento.

10. Recuperação de erros sintáticos — Slide 10

• Erro sintático: sequência de tokens incompatível com a gramática.

• Modo pânico: descartar tokens até um ponto seguro de sincronização, como ';' ou fechamento de bloco.

• Correção local: supor token ausente ou ignorar símbolo inesperado.

• Objetivo de mensagens úteis: indicar posição, o que foi encontrado e o que era esperado.

• Ênfase: escolher uma estratégia e explicar suas vantagens e riscos.

11. AST e código intermediário — Slide 10

• Diferença entre árvore sintática concreta e árvore sintática abstrata (AST).

• AST mantém operadores, operandos e construções essenciais, eliminando detalhes gramaticais desnecessários.

• Construção de AST para expressões e comandos de atribuição.

• Código intermediário e sua função entre programa-fonte e código-alvo.

• Código de três endereços e uso de temporários.

• Percurso da AST para preservar a ordem correta das operações.

• Ênfase: construir AST simples e gerar instruções temporárias para expressões.

Checklist final

☐ Consigo explicar por que $a^n b^n c^n$ exige um modelo mais forte que uma GLC?

☐ Consigo diferenciar linguagem recursiva de recursivamente enumerável e explicar decidibilidade?

☐ Consigo ordenar front-end, IR/middle-end e back-end e indicar a função de cada fase?

☐ Consigo diferenciar token, lexema e padrão e tokenizar uma linha curta?

☐ Consigo explicar por que scanners usam regex e autômatos e como funciona o maior lexema?

☐ Consigo diferenciar análise descendente de ascendente e explicar LL(1), FIRST/FOLLOW e recursão à esquerda?

☐ Consigo interpretar uma AST, respeitar precedência e gerar código de três endereços?

☐ Consigo propor uma estratégia de recuperação para erros léxicos e sintáticos?